QUEEN'S UNIVERSITY BELFAST

MSc ARTIFICIAL INTELLIGENCE

Pre-course preparation plan

13 July - 15 September 2026

A nine-week programme for Python fluency, mathematical refresh, classical machine learning and knowledge engineering.

Duration	Target workload	Core output	Course start
9 weeks + buffer	12-14 hours/week	Code portfolio + report	15 September 2026

GOAL **The central objective**

Arrive able to write, inspect and debug Python independently; use the scientific Python stack; recover the mathematics behind core AI methods; and complete a small, reproducible machine-learning investigation without outsourcing the reasoning to AI.

How this document is intended to work

- [ ] Print it or use the editable Word version as a weekly checklist.
- [ ] Follow the specified sections of each resource rather than trying to complete every course or textbook.
- [ ] Complete the minimum outcome first. Stretch work is optional and should never block the next week.
- [ ] Keep all code in one Git repository so progress is visible and recoverable.

Planning note

This guide is aligned to the current QUB course page and the latest accessible official programme specification. Module content and assessment arrangements can change; use the module handbooks issued at enrolment as the final authority.

Contents

Quick-start instructions

1. Today: create the repository, install a Python environment, and complete the diagnostic without AI.
2. Each Sunday: update the progress tracker, identify one recurring mistake, and choose the first task for Monday.
3. When time is tight: complete the minimum outcome and skip stretch work. Do not merge unfinished weeks together.
4. When stuck: use documentation, then request an explanation or hint from AI. Rebuild the solution from a blank file afterwards.

1. What to prioritise before September

The current programme structure places Foundations of AI, Machine Learning and Knowledge Engineering in the autumn term. The plan therefore prioritises the skills needed to survive those modules immediately. Computer Vision and Natural Language Processing are important, but deep learning receives only a short preview before the course begins.

Autumn area	Preparation needed	Your likely position	Priority
Python and scientific computing	Independent coding; testing; NumPy; pandas; plotting	Programming concepts transfer from JavaScript, but Python fluency is limited	**Very high**
Foundations of AI	Linear algebra; calculus; probability; optimisation; CSPs; MDPs	Strong engineering background, but likely rusty	**High**
Machine Learning	Model workflow; validation; metrics; classical algorithms; interpretation	Some data analysis experience, but limited formal ML	**Very high**
Knowledge Engineering	Logic; ontologies; knowledge graphs; Bayesian networks	Likely mostly new	**High**
Technical reporting	Reproducible experiments; critical discussion; limitations	A significant existing strength	**Maintain**
Deep learning	Tensors; training loop; automatic differentiation	Mostly new, but taught later	**Medium**

Target capability by 15 September

- [ ] Write and organise a small Python project without AI deciding its structure.
- [ ] Read, clean, transform and visualise tabular data using NumPy and pandas.
- [ ] Explain and apply the core mathematics behind linear models, PCA and optimisation.
- [ ] Build leak-free scikit-learn pipelines and evaluate models with defensible metrics.
- [ ] Understand the vocabulary of logic, ontologies, CSPs and MDPs well enough to follow postgraduate lectures.
- [ ] Produce a concise technical report that distinguishes results, interpretation, limitations and ethical risk.

> **IMPORTANT** **Do not optimise for course completion statistics**
>
> The goal is not to collect certificates or finish every video. A smaller number of concepts that you can reproduce, explain and apply independently will be far more useful than broad passive exposure.

2. Study method, weekly rhythm and AI guardrails

Recommended weekly rhythm

Use six flexible blocks rather than relying on a single long weekend session.

Block	Duration	Purpose
A	2 hours	New concept and worked examples
B	2 hours	Guided exercises
C	2 hours	Independent implementation
D	2 hours	Mathematics or theory problems
E	3-4 hours	Weekly project and tests
F	1-2 hours	Review, error log and no-AI retrieval

Structure of a two-hour session

- 15 minutes - retrieve the previous session from memory; do not open notes immediately.
- 35 minutes - learn one tightly defined concept.
- 60 minutes - solve problems or write code.
- 10 minutes - record mistakes, write the next task, and commit useful work.

AI-assisted coding protocol

1. Attempt the task for 25-30 minutes before asking AI for help.
2. Check official documentation before requesting generated code.
3. Ask for a hint, explanation, test case or code review rather than a full implementation.
4. After receiving help, close the answer and recreate the solution from a blank file.
5. Complete one 90-minute no-AI exercise each week. Documentation is allowed.
6. Record every material AI intervention in ai_log.md: the problem, the help requested, the misunderstanding and whether you can now reproduce the solution.

```
msc-ai-prep/
├── 01_python/
├── 02_numpy_pandas/
├── 03_linear_algebra/
├── 04_calculus_optimisation/
├── 05_probability_statistics/
├── 06_machine_learning/
├── 07_knowledge_engineering/
├── 08_advanced_ml/
├── 09_ai_foundations/
├── capstone/
├── notes/
└── ai_log.md
```

RECOVERY RULE **The rule for falling behind**

Do not extend a week indefinitely. Complete its minimum outcome, note the unfinished stretch items, and move on. The sequence matters more than polishing every exercise.

3. Baseline diagnostic

Complete this before using the week-by-week resources. Do not use AI.

A. Python diagnostic - 90 minutes

Write a command-line program that:

- [ ] reads a CSV containing experimental results;
- [ ] checks that required columns are present;
- [ ] reports or skips invalid rows;
- [ ] groups results by experimental condition;
- [ ] calculates count, mean, minimum and maximum for each group;
- [ ] writes the summary to a new CSV; and
- [ ] includes at least three automated tests.

First attempt: standard library only. Afterwards, note which parts were difficult because of Python syntax and which reflected a broader programming gap.

B. Mathematics diagnostic - 60 minutes

- [ ] Multiply two small matrices.
- [ ] Solve a two-variable linear system.
- [ ] Differentiate a polynomial and an exponential function.
- [ ] Find both partial derivatives of f(x, y) = x^2 + 3xy + y^2.
- [ ] Calculate a conditional probability using Bayes' theorem.
- [ ] Explain variance and covariance in your own words.

C. Machine-learning concepts - 30 minutes

- [ ] Define training, validation and test data.
- [ ] Define overfitting.
- [ ] Define regularisation.
- [ ] Define data leakage.
- [ ] Define classification versus regression.
- [ ] Define precision versus recall.
- [ ] Define cross-validation.

Score the result

Rating	Meaning	Action
Green	Solved and explained independently.	Follow the normal plan.
Amber	Understood after thought or checking notes.	Add one extra practice block.
Red	Did not know how to begin.	Shift 2-3 hours toward this area during the relevant week.

> OUTPUT **Diagnostic outcome to record**
>
> Write one sentence for each area: "I can already...", "I need to refresh...", and "I cannot yet...". Keep this page and repeat the diagnostic on 12 September.

Week 1: Python fundamentals and independent coding

Dates	Target time	Primary focus	End-of-week evidence
13-19 July	**12-14 hours**	Python fluency	Tested CSV experiment-summary CLI

Python is the working language for almost every later topic. This week is about replacing recognition of generated code with the ability to build and debug small programs yourself.

Learning targets

- [ ] Core syntax: variables, strings, lists, tuples, dictionaries and sets.
- [ ] Control flow: conditionals, loops and comprehensions.
- [ ] Functions, parameters, return values, scope and type hints.
- [ ] Exceptions, file I/O, imports, modules and virtual environments.
- [ ] Basic classes and dataclasses.
- [ ] Testing with pytest and reading tracebacks.

Resources - use these exact sections

Role	Resource	Use this part
Primary	CS50P - Introduction to Programming with Python	Use Weeks 0-6 selectively: Functions; Conditionals; Loops; Exceptions; Libraries; Unit Tests; File I/O.
Reference	The Python Tutorial	Use sections 3-9 when syntax or language behaviour is unclear.
Testing	pytest - Get Started	Follow the first test, exception testing and test discovery examples.

Build and practise

1. Create experiment_summary as a command-line program.
2. Separate parsing, calculations and output into modules.
3. Validate input and produce useful error messages.
4. Add at least six tests, including malformed data and an empty file.
5. Write a README with setup and usage instructions.

Minimum outcome

A working program split into functions, with at least three passing tests and no AI-generated implementation retained without reconstruction.

Stretch work

Add type hints throughout, a dataclass for a result row, and command-line options using argparse.

End-of-week self-check

- [ ] Create and activate a virtual environment.
- [ ] Explain mutable versus immutable objects.
- [ ] Choose appropriately between list, tuple, dictionary and set.
- [ ] Read a traceback and identify the failing line.
- [ ] Run pytest and interpret a failed assertion.

Week 2: NumPy, pandas and data analysis

Dates	Target time	Primary focus	End-of-week evidence
20-26 July	**12-14 hours**	Scientific Python	Reproducible exploratory-data-analysis notebook

Scientific Python changes how data problems are expressed. You need to become comfortable with arrays, tabular transformations and plotting before attempting machine learning.

Learning targets

- [ ] NumPy arrays, dimensions, shapes, dtypes and indexing.
- [ ] Boolean masks, reshaping, transposition and broadcasting.
- [ ] Vectorised operations and random-number generation.
- [ ] pandas Series and DataFrames; selection with loc and iloc.
- [ ] Missing values, grouping, aggregation, merging and derived columns.
- [ ] Clear plots and separation of exploration from reusable code.

Resources - use these exact sections

Role	Resource	Use this part
Primary	NumPy: Absolute Basics for Beginners	Complete array creation, indexing, shape, axes, basic operations and broadcasting.
Primary	pandas Getting Started Tutorials	Work through tutorials 1-6: tabular data, reading/writing, selection, plots, derived columns and statistics.
Reference	Matplotlib Quick Start Guide	Use for figures, axes, labels and saving plots.

Build and practise

1. Choose a small scientific or health-related CSV dataset.
2. Create a data-quality summary: shape, types, duplicates and missing values.
3. Produce descriptive statistics and at least four meaningful plots.
4. Write a short interpretation beneath each plot.
5. Move repeated cleaning logic into a Python module and call it from the notebook.

Minimum outcome

A notebook that runs from top to bottom, creates four labelled plots and states at least three limitations or potential confounders.

Stretch work

Write unit tests for the cleaning functions and export a cleaned dataset plus a machine-readable summary JSON.

End-of-week self-check

- [ ] Explain array shape and axis without looking them up.
- [ ] Use a Boolean mask and a groupby aggregation.
- [ ] Explain broadcasting with a small example.
- [ ] Distinguish loc from iloc.
- [ ] Explain why notebooks should not contain all reusable logic.

Week 3: Linear algebra for AI

Dates	Target time	Primary focus	End-of-week evidence
27 July-2 August	**12-14 hours**	Vectors and matrices	PCA from first principles notebook

Linear algebra is the language of data representation, transformations and many machine-learning algorithms. Prioritise geometric understanding as well as calculation.

Learning targets

- [ ] Scalars, vectors, matrices and tensors.
- [ ] Dot products, norms, distance and matrix multiplication.
- [ ] Systems of linear equations; rank, span, basis and independence.
- [ ] Orthogonality, projections and least squares intuition.
- [ ] Eigenvalues, eigenvectors and covariance matrices.
- [ ] Singular value decomposition and principal component analysis.

Resources - use these exact sections

Role	Resource	Use this part
Intuition	3Blue1Brown - Essence of Linear Algebra	Watch chapters 1-9 and the eigenvalue chapter. Pause and reproduce examples on paper.
Primary text	Mathematics for Machine Learning	Read Chapters 2, 3 and 4 selectively: linear algebra, analytic geometry and matrix decompositions.
Practice	MIT OCW - Linear Algebra	Use problem sets for systems, subspaces, orthogonality and eigenvalues when you need more practice.

Build and practise

1. Generate a correlated two-dimensional dataset.
2. Standardise the features.
3. Calculate its covariance matrix.
4. Find eigenvalues and eigenvectors with NumPy.
5. Project the data onto principal components.
6. Compare the result with scikit-learn PCA and explain any sign differences.

Minimum outcome

A correct two-dimensional PCA implementation with a written explanation of covariance, principal directions and information loss.

Stretch work

Generalise the implementation to n dimensions and plot explained variance for a higher-dimensional dataset.

End-of-week self-check

- [ ] Interpret matrix multiplication as a transformation.
- [ ] Explain span, basis and rank.
- [ ] Explain why eigenvectors are special directions.
- [ ] Connect covariance to PCA.
- [ ] State why feature scaling can change PCA results.

Schedule reset: 19 August 2026

The original plan placed calculus and optimisation on 3-9 August and probability and statistics on 10-16 August. The actual sequence changed: linear algebra/PCA took longer than planned, and the first probability-foundations notebook has now been completed before calculus. The remaining plan is reset from today rather than trying to squeeze missed weeks on top of the current week.

Current position

Area	Status	Evidence / note
Python fundamentals	Done	CLI, tests and environment work completed
NumPy / pandas / EDA	Done	Scientific-Python and EDA work completed
Linear algebra / PCA	Done, retrieval still needed	PCA from scratch completed; keep spaced retrieval
Probability foundations	Done	Events, distributions, expectation, variance, conditional probability and independence
Calculus / optimisation	Next	Originally scheduled earlier; now the immediate priority
Remaining probability / statistics	Pending	Bayes, base rates, normal distributions, likelihood and LLN
Classical ML / capstone	Pending	Very high priority before course start

Revised timeline

Dates	Target	Primary focus	End evidence
20-24 Aug	8-10 h	Calculus and optimisation	Gradient-descent linear regression
25-27 Aug	5-6 h	Probability completion	Bayes/base-rate + simulation notebook
28 Aug-3 Sep	12-14 h	Classical ML + capstone	First complete leak-free experiment
4-8 Sep	8-10 h	Knowledge engineering + CSP/MDP	Ontology + small symbolic-AI examples
9-12 Sep	8-10 h	Consolidation + capstone	Runnable repo + draft/final report
13 Sep	3 h	Final readiness assessment	Three-part no-AI assessment
14 Sep	Buffer	Setup, red items only, stop	Clean environment and rest

PLANNING GUARDRAIL

Retrieval can use older topics, but the new material in a session must follow this revised schedule unless you explicitly decide to change it. The next new-content session is calculus and optimisation. Probability should return only as retrieval until the 25-27 August block.

WHAT IS BEING DEPRIORITISED

Do not try to recover every original stretch item. Advanced Bayesian ML, a full unsupervised-learning comparison and a full PyTorch project become optional previews. Core calculus, probability, classical ML, symbolic-AI vocabulary and capstone reproducibility take priority.

20-24 August: Calculus and optimisation

Dates	Target time	Primary focus	End-of-block evidence
20-24 August	8-10 hours	Gradients and learning	Linear regression trained with gradient descent

You do not need proof-heavy calculus before September. The aim is to recover derivatives as rates of change, gradients as directions in parameter space, and optimisation as repeated parameter updates that reduce a loss.

Learning targets

- Differentiate polynomials, exponentials and simple composite functions.
- Use product and chain rules reliably.
- Calculate partial derivatives and a two-variable gradient.
- Interpret a gradient geometrically and connect its sign/magnitude to parameter updates.
- Understand local/global minima and convexity at an intuitive level.
- Define mean squared error and derive gradients for a one-feature linear-regression weight and bias.
- Implement batch gradient descent in NumPy and explain the learning-rate effect.
- Check an analytical gradient with finite differences.

Exercises - complete in order

1. Derivative refresh - On paper, differentiate 6 short functions: two polynomials, one exponential, two chain-rule examples and one product-rule example. Output: working plus derivative for each.
2. Partial derivatives and gradient - For a supplied two-variable loss surface, calculate both partial derivatives and the gradient at 3 specified points. Output: gradient vector at each point plus one sentence saying which way would reduce the function.
3. Linear-regression derivation - Write the MSE for y-hat = wx + b and derive dL/dw and dL/db on paper. Output: one-page derivation that you can explain line by line.
4. NumPy gradient descent - Generate a small regression dataset; implement prediction, MSE, gradient calculation and parameter updates. Output: learned w and b plus a loss history array.
5. Learning-rate comparison - Run the same fit with three learning rates. Output: one loss-vs-iteration plot with labelled curves and a short diagnosis of slow convergence, useful convergence or divergence.
6. Gradient check - Use a small finite-difference perturbation to check both analytical gradients. Output: analytical vs numerical values and an allclose-style check.

Minimum outcome	Stretch only if ahead
A working NumPy gradient-descent implementation, the linear-regression gradient derivation, and a clear explanation of learning rate and feature scaling.	Mini-batch gradient descent, Hessian/Jacobian detail, or extra optimisation methods. Do not let these delay the next block.

RETRIEVAL	Start one calculus session with a 10-minute no-notes check on probability definitions: random variable, expectation, variance, P(A and B) versus P(A given B), and independence. Do not turn that retrieval into new probability teaching.

25-27 August: Probability and statistics completion

Dates	Target time	Primary focus	End-of-block evidence
25-27 August	5-6 hours	Bayesian reasoning and statistical intuition	Bayes/base-rate + simulation notebook

Probability foundations are already in place. This block does not repeat the completed dice work; it extends it into the ideas that are most useful for classification, model uncertainty and later Bayesian reasoning.

Already covered

- Sample spaces, events, union/intersection/complement.
- Random variables and discrete probability distributions.
- Expectation, variance and standard deviation.
- Joint and conditional probability.
- Independence and the test P(A and B) = P(A)P(B).

New learning targets

- Bayes theorem and diagnostic-test base rates.
- Prior, likelihood and posterior as distinct ideas.
- Normal distribution: mean, variance and standard deviation.
- Covariance versus correlation, including the effect of units/scale.
- Law of large numbers through simulation.
- Likelihood and maximum-likelihood intuition; no proof-heavy derivation.
- Gaussian Naive Bayes intuition as a bridge into the ML block.

Exercises - clear outputs

1. Bayes by counts - Use a hypothetical 10,000-person diagnostic-testing population with low prevalence, sensitivity and specificity. Output: a 2x2 count table and P(disease | positive) calculated from counts.

2. Bayes by formula - Solve the same problem with Bayes theorem. Output: formula substitution and a check that it matches Exercise 1.

3. Base-rate simulation - Simulate the diagnostic test with NumPy. Output: simulated posterior probability and a short explanation of why false positives can dominate at low prevalence.

4. Normal distributions - Plot three normal distributions that vary mean or standard deviation one change at a time. Output: labelled plot plus two sentences interpreting location and spread.

5. Covariance vs correlation - Create two linearly related variables, then rescale one by 100. Output: covariance and correlation before/after scaling plus an explanation of what changed and what did not.

6. Law of large numbers - Simulate cumulative means from repeated coin tosses or dice rolls. Output: cumulative-mean plot and a one-paragraph interpretation.

Minimum outcome	Bridge to ML
A reproducible Bayes/base-rate simulation notebook and a written explanation of prior, likelihood, posterior and why P(A|B) differs from P(B|A).	Gaussian Naive Bayes may be introduced at the end, but the from-scratch classifier can move into the classical-ML block if time is tight.

28 August-3 September: Classical machine learning and capstone

Dates	Target time	Primary focus	End-of-block evidence
28 Aug-3 Sep	12-14 hours	Reliable ML workflow	First complete capstone experiment

This remains the highest-priority applied block. The objective is a defensible experiment, not memorising every algorithm. The capstone should become the vehicle for learning train/test splitting, preprocessing, cross-validation, metrics and model comparison.

Learning targets

- Features, targets, baselines and held-out test data.
- Preprocessing pipelines and data leakage.
- Cross-validation only on the training data.
- Logistic regression, k-nearest neighbours, SVM and random forest at an explainable level.
- Optional Gaussian Naive Bayes to connect the probability block.
- Regularisation and bias-variance intuition.
- Confusion matrix; accuracy, precision, recall, specificity, F1 and ROC-AUC.
- Hyperparameter tuning without touching the final test set.
- Reproducible seeds, clean notebook/script execution and result interpretation.

Capstone exercises

1. Define task - Load Breast Cancer Wisconsin data, identify X/y, state the prediction task and state what the project does not claim.
2. Hold out test set - Create one final test split before model comparison. Output: sizes and class balance; no test-set model selection.
3. Baseline - Fit a DummyClassifier. Output: baseline metrics and one sentence explaining why the baseline matters.
4. Pipelines - Build pipelines so scaling/preprocessing occurs inside CV. Output: at least logistic regression and k-NN pipelines.
5. Cross-validation - Compare at least three real models plus dummy on training data only. Output: mean and spread of chosen CV metrics.
6. Metric choice - Choose one primary and one secondary metric and justify them for this educational health-related task.
7. Final test - Select the model using training/CV evidence, then evaluate once on untouched test data. Output: test metrics and confusion matrix.
8. Error analysis - Inspect mistakes and write plausible explanations plus dataset/deployment limitations.

Minimum outcome	Do not spend time on
A clean rerunnable experiment comparing a dummy baseline with at least three real models, with preprocessing inside pipelines and no test leakage.	Nested CV, calibration, permutation importance or exhaustive tuning until the minimum experiment is complete.

4-8 September: Knowledge engineering and AI foundations

Dates	Target time	Primary focus	End-of-block evidence
4-8 September	8-10 hours	Symbolic AI + problem formulation	Small ontology, CSP and MDP example

The original plan separated knowledge engineering from CSP/MDP work. They are now combined into one focused block because both are high-value vocabulary for the autumn modules, while complete mastery is not required before enrolment.

Part A - logic and ontologies

- Propositions, truth tables, implication and equivalence.
- Validity, satisfiability and entailment at an introductory level.
- Predicates, variables and quantifiers.
- Classes, instances, properties and restrictions.
- Open-world versus closed-world assumptions.
- What an ontology adds beyond a spreadsheet.

Part B - CSPs and MDPs

- CSP variables, domains and constraints; backtracking intuition.
- MDP states, actions, transitions, rewards and policies.
- Discount factor and state-value intuition.
- Bellman update and value iteration at a small tabular scale.

Exercises - complete the minimum versions

1. Logic sheet - Construct two truth tables and translate four short English statements into propositional or predicate notation. Output: worked answers.
2. Protégé ontology - Model a small diagnostic domain using classes such as DiagnosticTest, SampleType, Pathogen, AssayMethod and Instrument. Output: saved ontology with hierarchy, properties and one restriction.
3. Reasoning check - Create one deliberate inconsistency or inference and use a reasoner. Output: screenshot/note explaining what was detected or inferred.
4. CSP - Implement map colouring or a tiny timetable allocation problem. Output: variables/domains/constraints stated explicitly and a valid solution.
5. MDP - Work through or implement a tiny grid-world value-iteration example. Output: states/actions/rewards/discount factor plus at least several value updates and the resulting policy intuition.

Minimum outcome	Stretch only
A coherent small ontology, one CSP you can formulate clearly, and an MDP/value-iteration example you can explain at a high level.	SPARQL/RDF code, sophisticated ontology design, advanced CSP heuristics or policy visualisation.

9-12 September: Consolidation, capstone and targeted preview

Dates	Target time	Primary focus	End-of-block evidence
9-12 September	8-10 hours	Consolidate core readiness	Runnable capstone + report + targeted retrieval

This block replaces the original full advanced-ML week. By this point, consolidation is more valuable than opening several large new topics. Advanced material becomes conditional on the core checklist being in good shape.

Core tasks - do these first

1. Capstone report - Write/finalise the 1,500-2,000 word report: problem/context, data, methods, results, error analysis, limitations/ethics and conclusion.
2. Reproducibility pass - Create a clean environment from documented dependencies and rerun the capstone from the README.
3. Mixed retrieval - Complete one 90-minute no-AI block covering Python, NumPy shapes, linear algebra/PCA, calculus/gradients, probability/Bayes and ML workflow.
4. Red-item repair - Choose only the items that are genuinely red. Do one focused exercise per red item rather than reopening whole topics.
5. Module vocabulary - Review logic/ontology/CSP/MDP terminology so lecture vocabulary is familiar.

Optional preview - only after core tasks are complete

• PyTorch: 60-90 minutes on tensors, a tiny model, loss.backward(), optimiser.step(), and the purpose of autograd.

• Clustering: read the scikit-learn overview and compare k-means vs one alternative on a small dataset; do not build a large project.

• Bayesian ML: vocabulary-only skim of Bayesian linear regression, Gaussian processes and MCMC; no derivations or implementation required.

DECISION RULE	If the capstone is not rerunnable, calculus/Bayes are still red, or the symbolic-AI vocabulary is unfamiliar, skip the optional preview. Deep learning was only a medium priority in the original plan; it should not displace core readiness now.

Minimum outcome	Useful evidence
Capstone runs cleanly, report is complete or nearly complete, and no core readiness item remains red without a written recovery task.	README, requirements file, final results table/plots, report draft, retrieval answers and a short unresolved-questions list.

13-14 September: Final readiness assessment, setup and stop

13 September - three-hour no-AI assessment

Time	Task	Evidence
60 min	Python data-processing task from an unfamiliar CSV: validate, transform, summarise and test.	Working files + tests
60 min	Paper mathematics: matrices/PCA, partial derivatives/gradient, Bayes theorem and covariance.	Typed or scanned answers
60 min	Train/evaluate a basic ML pipeline from unfamiliar data; justify split, preprocessing and metrics.	Runnable notebook/script

13 September - scoring

• Green: solved and explained independently.

• Amber: understood after thought or documentation; note one retrieval task.

• Red: could not begin or explanation remains confused; this is the only material eligible for 14 September review.

14 September - buffer, setup and stop

• Repeat only red items from the assessment.

• Confirm Python, VS Code, Jupyter and Git all use the intended environment.

• Rerun the capstone from a clean environment and its README.

• Push the final preparation repository and make a backup.

• Organise notes by Foundations, Machine Learning and Knowledge Engineering.

• Write a one-page unresolved-questions list for the first teaching week.

• Stop at a reasonable time. Do not turn the final day into an all-day cram.

PERSPECTIVE	Readiness does not mean every item is effortless. The strongest target is green on Python workflow and core ML, with amber rather than red on mathematics and symbolic-AI vocabulary. The aim is to make lectures intelligible and productive, not to pre-complete the MSc.

5. Capstone project specification

A small, rigorous project is more valuable than a large, unfinished one.

Recommended question

How do several classical classification models compare on the scikit-learn Breast Cancer Wisconsin dataset when evaluated using a leak-free, reproducible workflow?

This is an educational exercise, not a clinically valid diagnostic study. The dataset is small, historical and unsuitable for medical deployment claims.

Required models

- [ ] Dummy classifier baseline
- [ ] Logistic regression
- [ ] k-nearest neighbours
- [ ] Support-vector machine
- [ ] Random forest

Required workflow

1. Define the prediction task and target clearly.
2. Create a held-out test set before model comparison.
3. Use pipelines for scaling or other preprocessing.
4. Perform cross-validation only on the training data.
5. Choose and justify primary and secondary metrics.
6. Report uncertainty or variability across folds, not only a single score.
7. Compare final models on the untouched test set once.
8. Inspect errors and discuss plausible causes.
9. State dataset limitations, fairness concerns and deployment risks.
10. Make the repository runnable from a clean environment.

Report structure - 1,500 to 2,000 words

Section	Purpose	Suggested length
1. Problem and context	Define the task, intended use and what the project does not claim.	150-200
2. Data	Describe samples, features, target, data quality and limitations.	200-250
3. Methods	Explain split strategy, preprocessing, models, tuning and metrics.	350-450
4. Results	Present cross-validation and test results with clear tables/plots.	300-400
5. Error analysis	Examine mistakes and model behaviour.	200-250
6. Limitations and ethics	Discuss sample size, representativeness, leakage, fairness and clinical risk.	250-350
7. Conclusion	State what was learned and what should be done next.	100-150

Repository acceptance test

- [ ] A new user can install dependencies from documented instructions.
- [ ] Random seeds are controlled where relevant.
- [ ] The code creates results without manual notebook state.
- [ ] Plots have titles, labels and readable legends.
- [ ] No preprocessing is fitted on the full dataset before cross-validation.
- [ ] The README states the educational and non-clinical nature of the work.

6. Progress tracker - revised 19 August

Evidence should be a file path, commit, notebook or report section. The tracker below reflects the schedule reset and distinguishes work already completed from the remaining milestones.

Milestone	Evidence / file	Status
Baseline diagnostic	notes / diagnostic work	DONE
Python fundamentals + CLI	01_python/	DONE
NumPy / pandas / EDA	02_numpy_pandas/	DONE
Linear algebra + PCA	03_linear_algebra/	DONE - retrieve later
Probability foundations	04_probability/01_probability_foundations.ipynb	DONE
Calculus + gradient descent	05_calculus_optimisation/	NEXT
Bayes + probability completion	04_probability/	PENDING
Classical ML + capstone experiment	06_machine_learning/ + capstone/	PENDING
Knowledge engineering + CSP/MDP	07_knowledge_engineering/ + 08_ai_foundations/	PENDING
Capstone report + consolidation	capstone/report.* + README	PENDING
Optional PyTorch / clustering preview	09_advanced_ml/ (optional)	STRETCH

Session reflection

Date / block	Most important thing learned	Recurring error or misconception	First task next session
20-24 Aug - Calculus			
25-27 Aug - Probability			
28 Aug-3 Sep - ML/capstone			
4-8 Sep - KE + AI foundations			
9-12 Sep - Consolidation			

Revised workload tracker and lesson-planning rules

The remaining schedule is intentionally compressed. Record actual hours, but do not compensate for a short block by automatically stealing time from the next high-priority block.

Block	Target	Actual	No-AI practice?	Adjustment
20-24 Aug - Calculus	8-10 h		Yes / No	
25-27 Aug - Probability	5-6 h		Yes / No	
28 Aug-3 Sep - ML/capstone	12-14 h		Yes / No	
4-8 Sep - KE + AI foundations	8-10 h		Yes / No	
9-12 Sep - Consolidation	8-10 h		Yes / No	
13 Sep - Final assessment	3 h		No AI	

Lesson-planning rules from this revision

• At the start of a session, check the current date/block before choosing new material.

• Retrieval questions may come from any completed topic; retrieval does not change the scheduled new topic.

• Teach new concepts before testing them. Retrieval questions must only cover previously taught material.

• Exercises must state the input/context, exact task and expected output.

• When an exercise reveals a gap, repair that gap locally rather than silently switching the whole session to another subject.

• Use the minimum outcome as the stopping rule. Stretch work is optional and must not delay the next block.

• PCA and probability should return through spaced retrieval while calculus and ML progress.

• On 9 September, reassess what remains red. Optional PyTorch/clustering work only happens if the core list is in good shape.

NEXT SESSION	Calculus and optimisation. Start with a short retrieval check from probability/PCA, then move into derivatives and gradients. Do not continue into new Bayes material until the dedicated probability-completion block.

7. Final readiness assessment

Complete on 12 or 13 September using documentation but no AI-generated code.

Three-hour assessment

Time	Task	Evidence
60 min	Python data-processing task from an unfamiliar CSV: validate, transform, summarise and test.	Working files and tests.
60 min	Paper mathematics: matrices, partial derivatives, gradient, Bayes theorem, covariance and one PCA explanation.	Scanned or typed answers.
60 min	Train and evaluate a basic ML pipeline from an unfamiliar CSV; justify the split, preprocessing and metrics.	Runnable notebook or script.

Readiness checklist

- [ ] Write a 150-300 line Python project without AI deciding its structure.
- [ ] Use virtual environments, modules, tests and Git.
- [ ] Load, clean, transform and visualise tabular data.
- [ ] Work comfortably with NumPy shapes and matrix operations.
- [ ] Explain eigenvectors, gradients, Bayes' theorem and covariance.
- [ ] Implement linear regression and gradient descent using NumPy.
- [ ] Build a leak-free scikit-learn pipeline.
- [ ] Compare models with cross-validation and suitable metrics.
- [ ] Explain overfitting, regularisation and bias-variance.
- [ ] Construct basic logical statements and a small OWL ontology.
- [ ] Explain the components of a CSP and an MDP.
- [ ] Train a simple PyTorch network and explain its training loop.
- [ ] Produce a reproducible technical report with limitations and ethical considerations.

> TARGET **How to interpret the result**
>
> You do not need every item to feel effortless. Green on Python workflow and core ML, with amber rather than red on the mathematics and symbolic-AI vocabulary, is a strong starting position.

8. Resource directory

All links are clickable in the Word and PDF versions.

Topic	Resource	Purpose
Course information	QUB MSc Artificial Intelligence course page	Official overview, course structure, modules and entry requirements.
Course information	QUB official programme specification	Current accessible formal programme aims, learning outcomes, module terms and assessment pattern.
Python	CS50P	Structured Python lessons and exercises.
Python	Official Python Tutorial	Language reference for programmers new to Python.
Testing	pytest Get Started	Testing basics and test discovery.
Scientific Python	NumPy Absolute Basics	Arrays, shapes, indexing, operations and broadcasting.
Scientific Python	pandas Intro Tutorials	DataFrames, files, selection, plots and aggregation.
Plotting	Matplotlib Quick Start	Figure and axes fundamentals.
Mathematics	Mathematics for Machine Learning	Linear algebra, calculus, probability and optimisation in one free text.
Linear algebra	3Blue1Brown Essence of Linear Algebra	Visual intuition for vectors, transformations and eigenvectors.
Linear algebra	MIT OCW Linear Algebra	Lectures, notes and problem sets.
Calculus	3Blue1Brown Essence of Calculus	Visual intuition for derivatives, chain rule and gradients.
Calculus	MIT OCW Multivariable Calculus	Partial derivatives, gradients and optimisation practice.
Probability	MIT OCW Probability and Statistics	Conditioning, distributions, inference and confidence intervals.
Probability	Seeing Theory	Interactive visual probability and statistics.
Machine learning	scikit-learn MOOC	Practical modelling workflow, validation, pipelines and algorithms.
Machine learning	scikit-learn Common Pitfalls	Leakage, preprocessing and reproducibility.
Unsupervised ML	scikit-learn Unsupervised Learning Guide	Clustering and dimensionality reduction.
Knowledge engineering	W3C OWL 2 Primer	Formal introduction to OWL ontologies.
Knowledge engineering	Protégé	Ontology editor and reasoning environment.
Classical AI	AIMA resources	Logic, CSP, probabilistic reasoning and decision-making references.

Topic	Resource	Purpose
Classical AI	AIMA Python	Reference implementations for CSPs, MDPs and other AI algorithms.
Deep learning	PyTorch Learn the Basics	Tensors, models, autograd and optimisation.
Bayesian ML	Probabilistic Machine Learning	Free advanced reference; skim only before the course.
Reproducibility	The Turing Way	Practical reproducible-research guidance.

Resource discipline

- [ ] Use the primary resource named in each week first.
- [ ] Use reference material only to resolve a specific confusion.
- [ ] Do not add a new course because the current explanation feels difficult; attempt exercises and seek a second explanation only after identifying the exact gap.
- [ ] Save useful links and notes in the repository README so they are available when the MSc begins.

Build fluency first. Then build depth.

The purpose of this plan is to make the opening weeks of the MSc challenging but intelligible - not to complete the MSc before it starts.